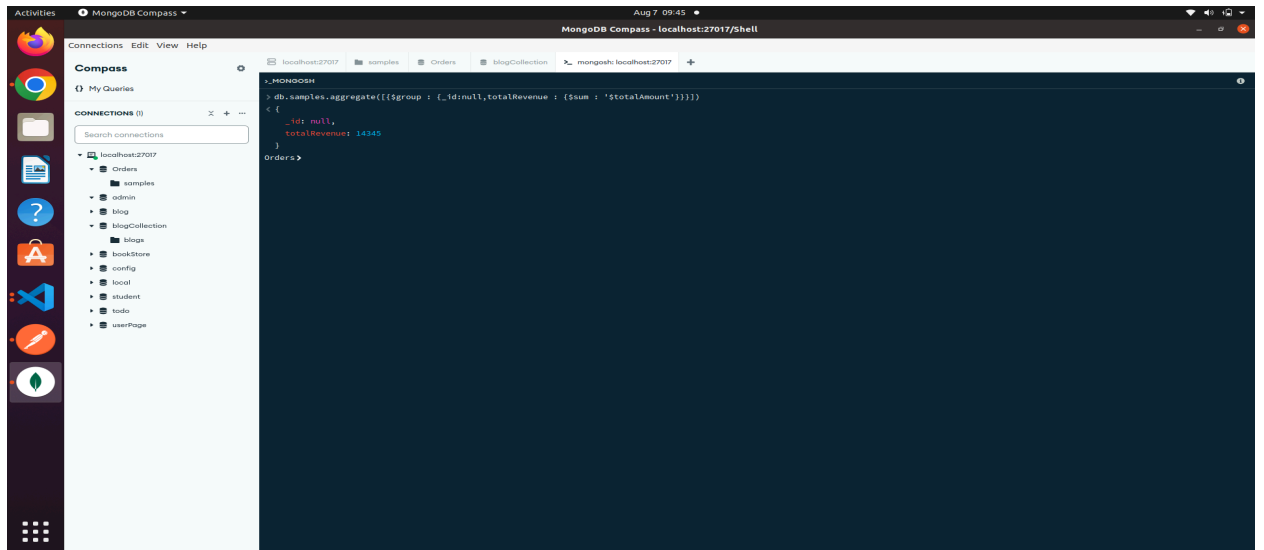


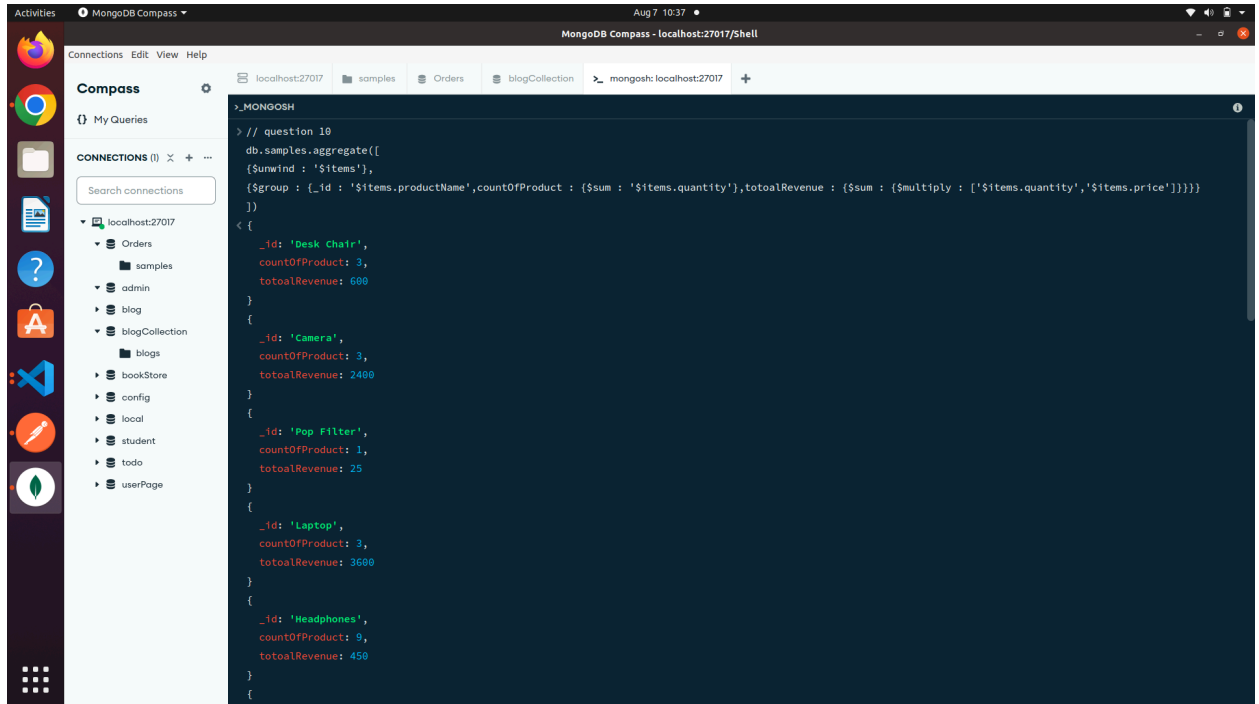
MongoDb Assessment

Aggregation :

1. Find total revenue generated (sum of totalAmount).

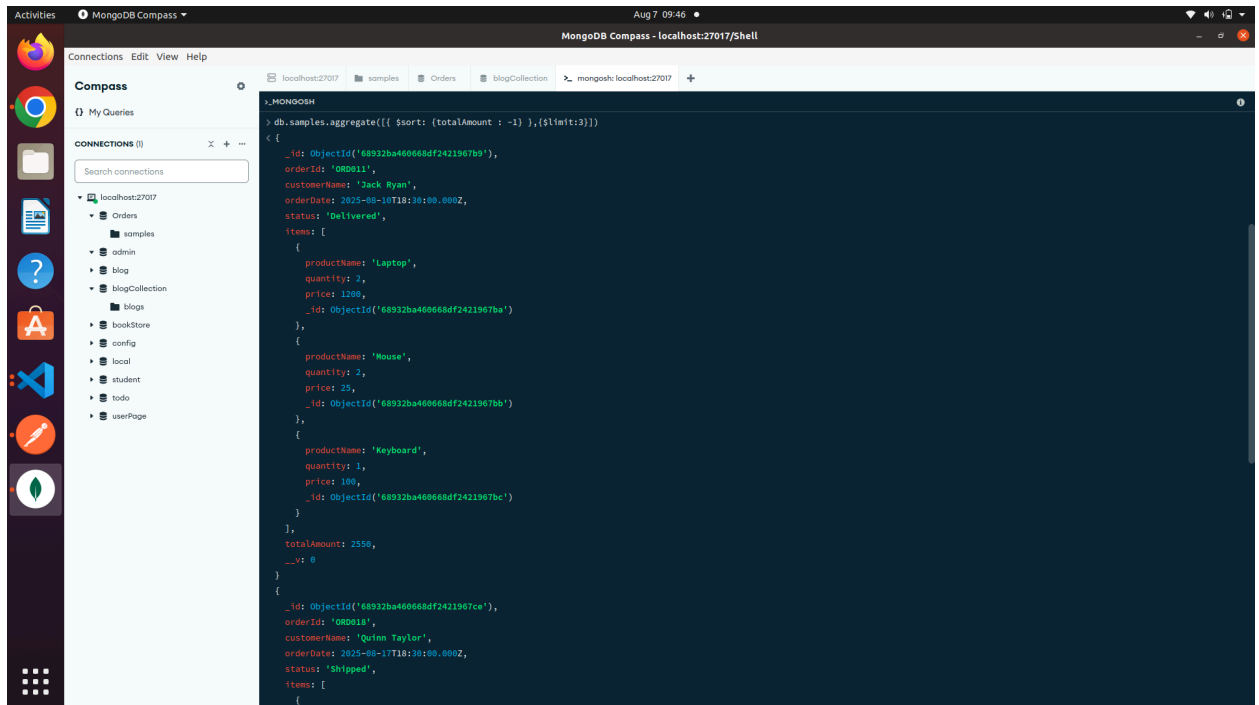


2. Find total number of orders by status (Pending, Shipped, Deliver)



```
> // question 10
db.samples.aggregate([
  {$unwind : '$items'},
  {$group : {$_id : '$items.productName',countOfProduct : {$sum : '$items.quantity'},totalRevenue : {$sum : {$multiply : ['$items.quantity','$items.price']}}}}
])
<
[
  {
    _id: 'Desk Chair',
    countOfProduct: 3,
    totalRevenue: 600
  },
  {
    _id: 'Camera',
    countOfProduct: 3,
    totalRevenue: 2400
  },
  {
    _id: 'Pop Filter',
    countOfProduct: 1,
    totalRevenue: 25
  },
  {
    _id: 'Laptop',
    countOfProduct: 3,
    totalRevenue: 3600
  },
  {
    _id: 'Headphones',
    countOfProduct: 9,
    totalRevenue: 450
  }
]
```

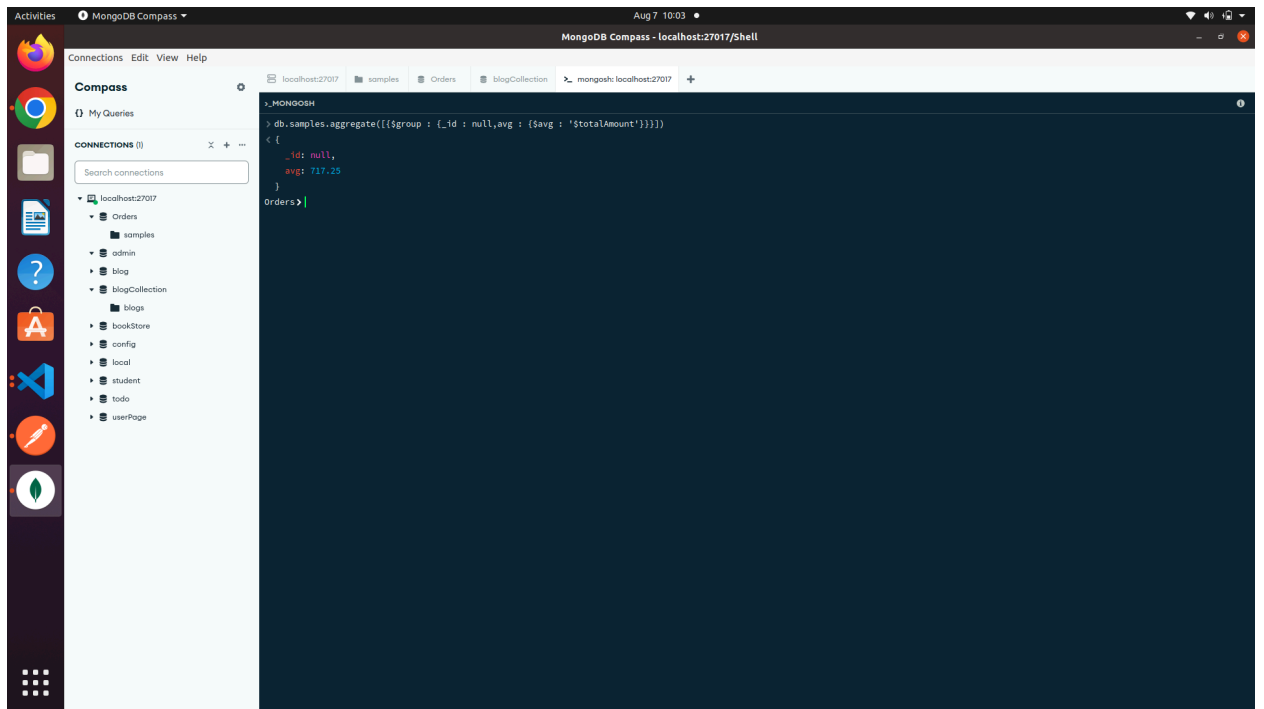
3. Find the top 3 customers who spent the most (sort by totalAmount).



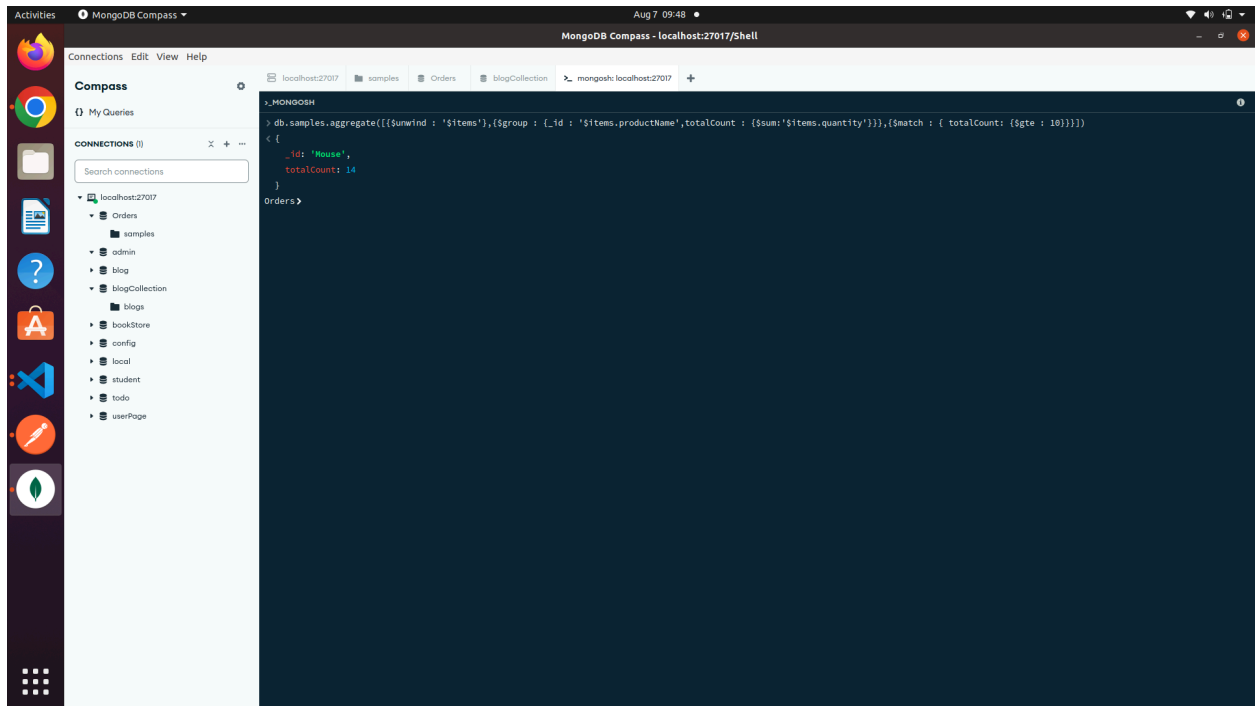
```
> db.samples.aggregate([{$sort : {totalAmount : -1 } },{$limit:3}])
<
[
  {
    _id: ObjectId('68932ba460668df2421967b9'),
    orderId: 'ORD001',
    customerName: 'Jack Ryan',
    orderDate: 2025-08-10T18:10:00.000Z,
    status: 'Delivered',
    items: [
      {
        productName: 'Laptop',
        quantity: 2,
        price: 1200,
        _id: ObjectId('68932ba460668df2421967ba')
      },
      {
        productName: 'Mouse',
        quantity: 2,
        price: 25,
        _id: ObjectId('68932ba460668df2421967bb')
      },
      {
        productName: 'Keyboard',
        quantity: 1,
        price: 100,
        _id: ObjectId('68932ba460668df2421967bc')
      }
    ],
    totalAmount: 2550,
    __v: 0
  },
  {
    _id: ObjectId('68932ba460668df2421967ce'),
    orderId: 'ORD018',
    customerName: 'Quinn Taylor',
    orderDate: 2025-08-11T18:10:00.000Z,
    status: 'Shipped',
    items: [

```

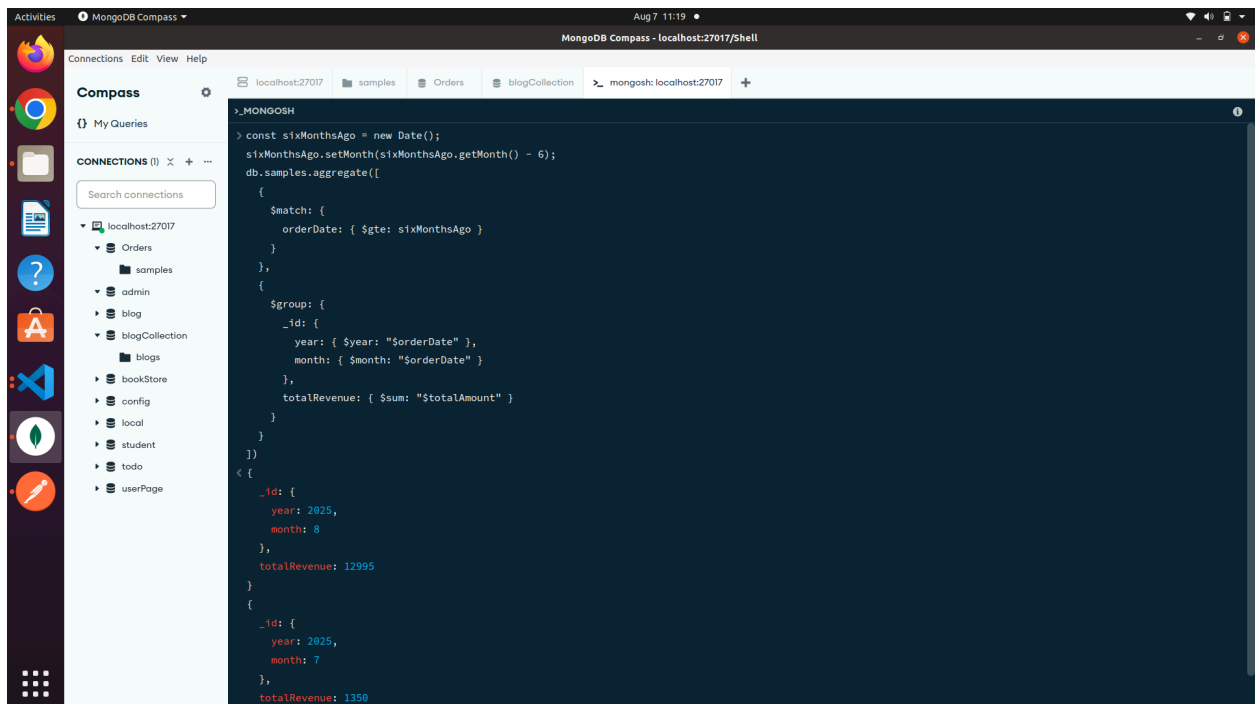
4. Get the average order amount per customer.



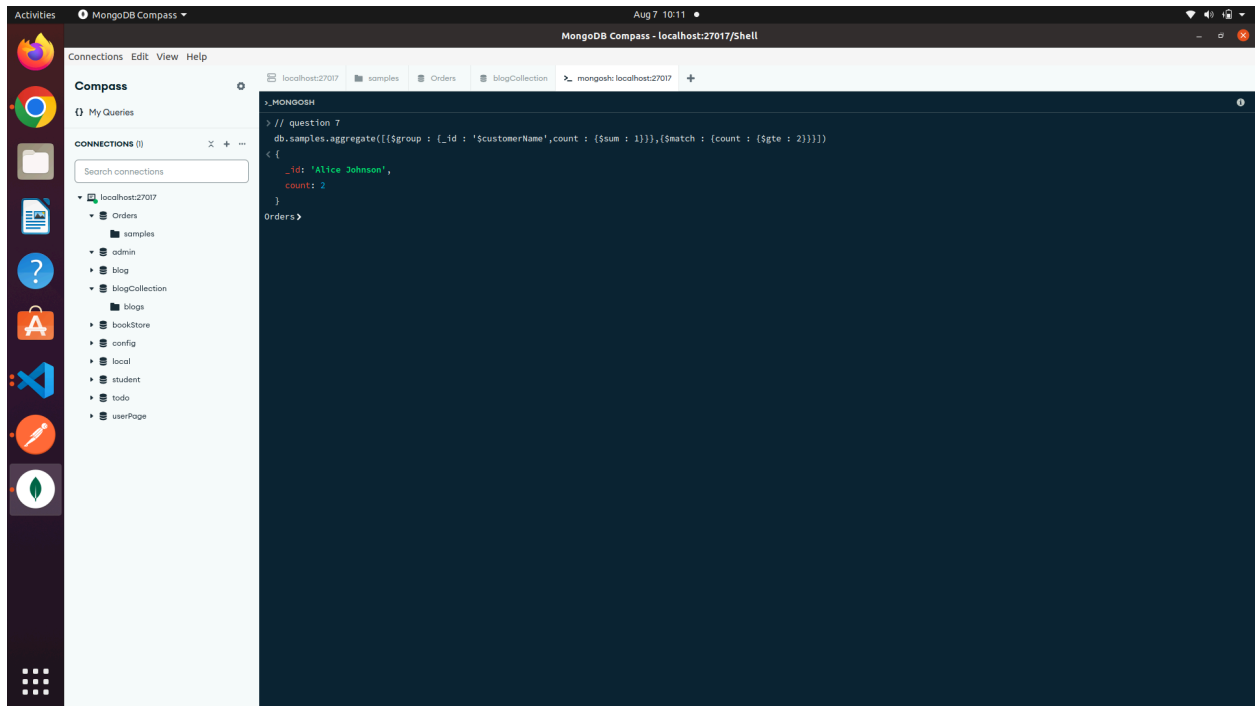
5. Find products that were sold more than 10 times (total quantity).



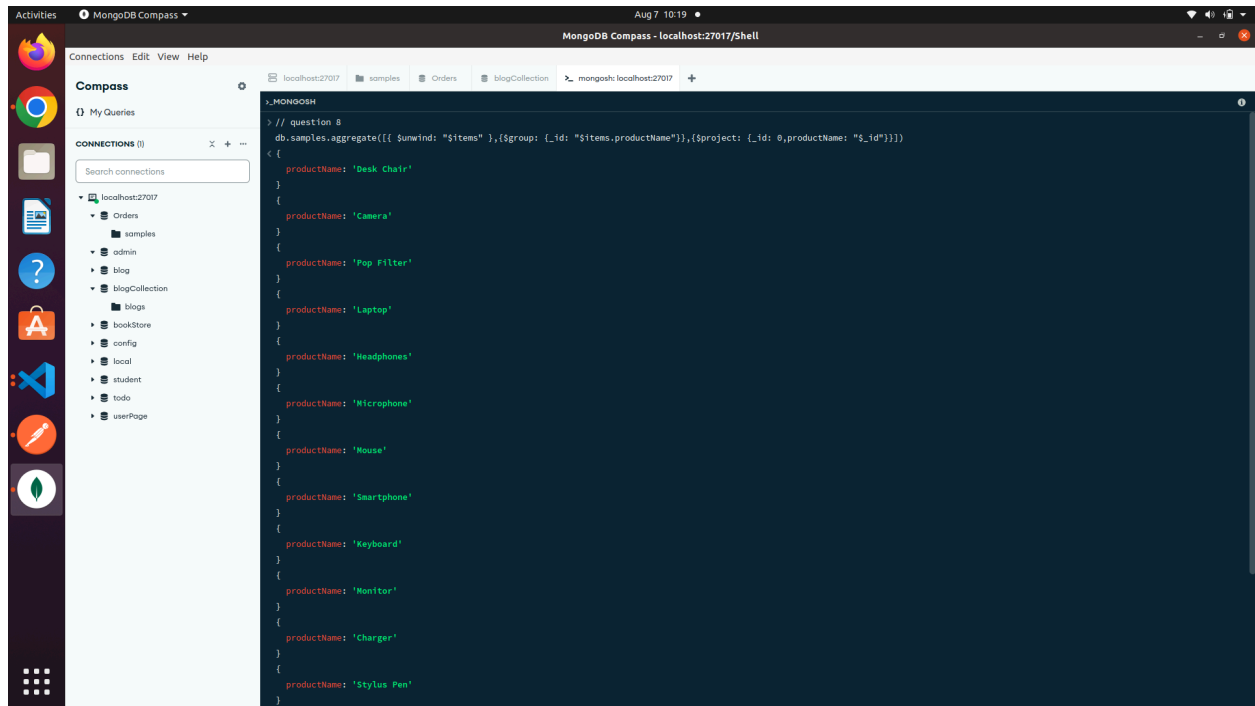
6. List monthly revenue (group by month-year) for the last 6 months.



7. Find all customers who placed more than 2 orders.



8. Extract only the product names from all orders using \$unwind and \$project.

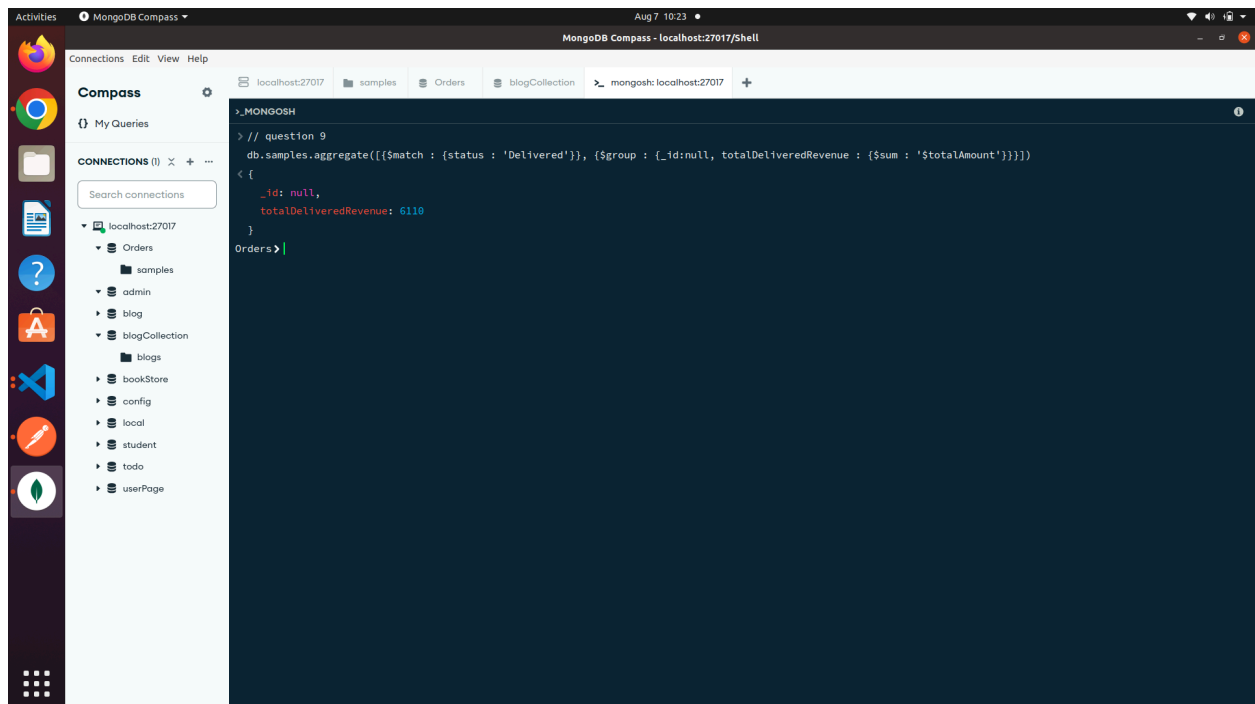


The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' panel with a search bar and a list of databases. The 'localhost:27017' database is selected, showing collections: 'Orders', 'samples', 'admin', 'blog', 'blogCollection', 'blogs', 'bookStore', 'config', 'local', 'student', 'todo', and 'userPage'. The 'Orders' collection is selected. The main editor shows a query in the 'MONGOSH' shell:

```
> // question 8
db.samples.aggregate([{$unwind: '$items' },{$group: {_id: '$items.productName'},{$project: {_id: 0,productName: '$_id'}}])
```

The result is displayed as a JSON array of objects, each containing a 'productName' field with values: 'Desk Chair', 'Camera', 'Pop Filter', 'Laptop', 'Headphones', 'Microphone', 'Mouse', 'Smartphone', 'Keyboard', 'Monitor', 'Charger', and 'Stylus Pen'.

9. Apply filtering using \$match (only Delivered orders) and then calculate revenue.

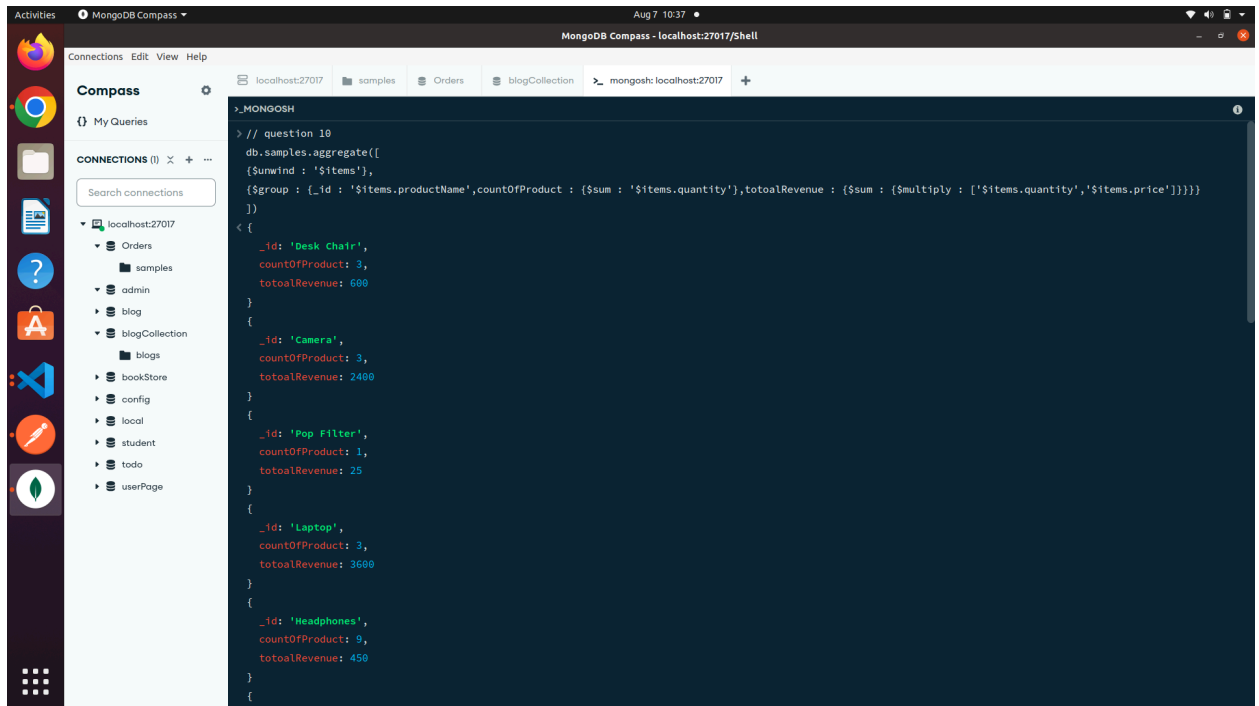


The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' panel with a search bar and a list of databases. The 'localhost:27017' database is selected, showing collections: 'Orders', 'samples', 'admin', 'blog', 'blogCollection', 'blogs', 'bookStore', 'config', 'local', 'student', 'todo', and 'userPage'. The 'Orders' collection is selected. The main editor shows a query in the 'MONGOSH' shell:

```
> // question 9
db.samples.aggregate([{$match: {status: 'Delivered'}}, {$group: {_id: null, totalDeliveredRevenue: {$sum: '$totalAmount'}}}])
```

The result is displayed as a JSON array of objects, each containing a '_id' field with value 'null' and a 'totalDeliveredRevenue' field with value '6110'.

10. Calculate total quantity and total revenue per product (use \$unwind and \$group).



The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel shows a connection to 'localhost:27017'. The 'Collections' panel on the right lists several collections: 'Orders', 'samples', 'admin', 'blog', 'blogCollection', 'blogs', 'bookStore', 'config', 'local', 'student', 'todo', and 'userPage'. The main panel displays an aggregation query in the 'MONGOSH' shell:

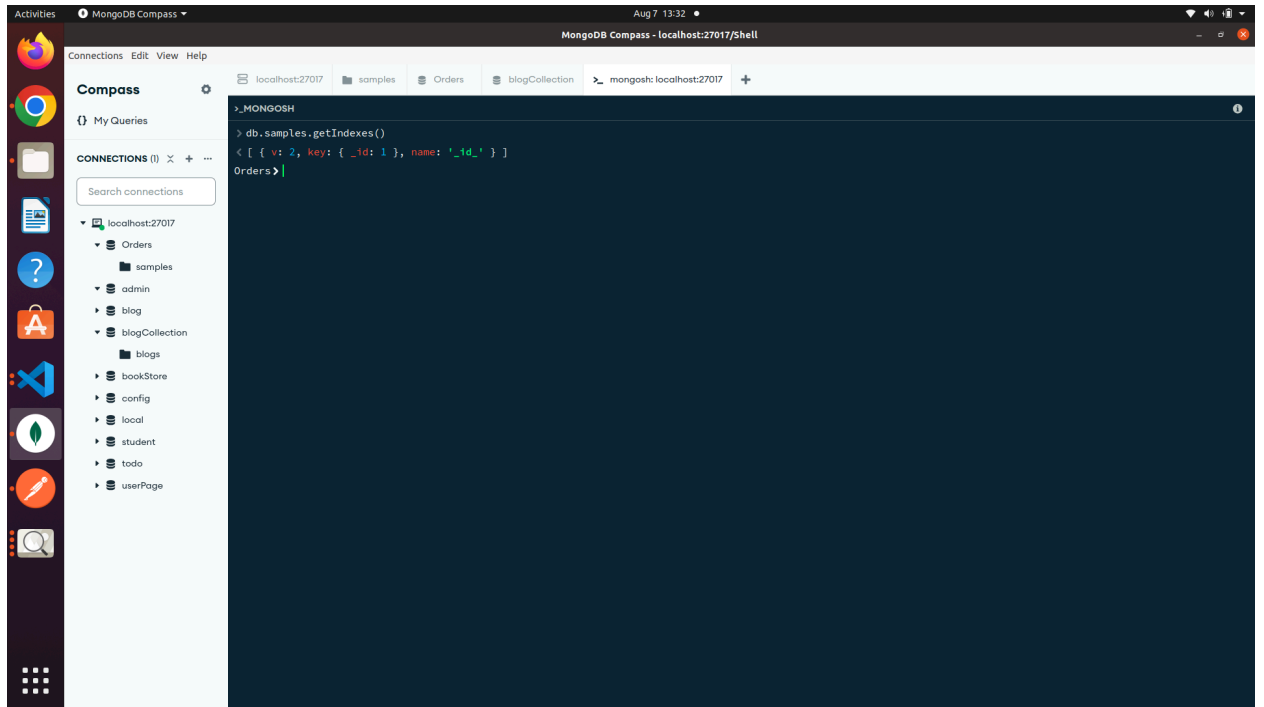
```
> // question 10
db.samples.aggregate([
  {$unwind : '$items'},
  {$group : {_id : '$items.productName',countOfProduct : {$sum : '$items.quantity'},totalRevenue : {$sum : {$multiply : ['$items.quantity','$items.price']}}}}
])
```

The results of the aggregation are shown below the query:

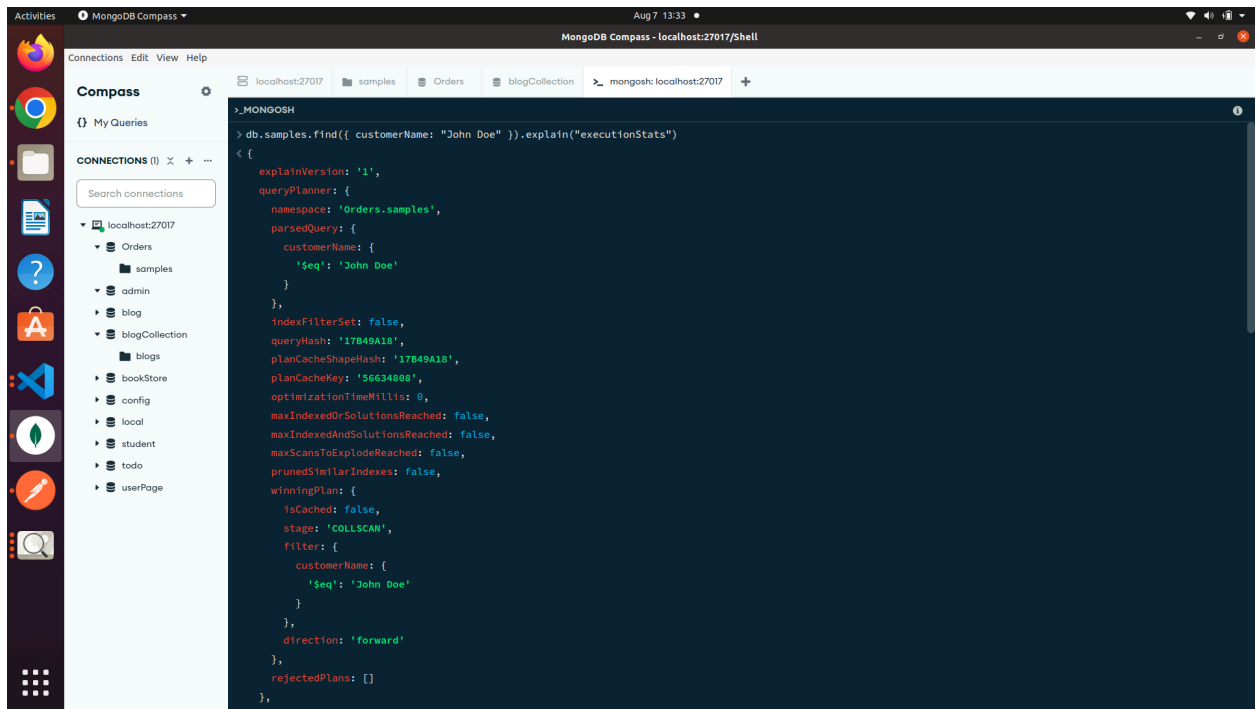
```
< {
  _id: 'Desk Chair',
  countOfProduct: 3,
  totalRevenue: 600
}
{
  _id: 'Camera',
  countOfProduct: 3,
  totalRevenue: 2400
}
{
  _id: 'Pop Filter',
  countOfProduct: 1,
  totalRevenue: 25
}
{
  _id: 'Laptop',
  countOfProduct: 3,
  totalRevenue: 3600
}
{
  _id: 'Headphones',
  countOfProduct: 9,
  totalRevenue: 450
}
```

Indexing :

1. Check indexes on the collection.

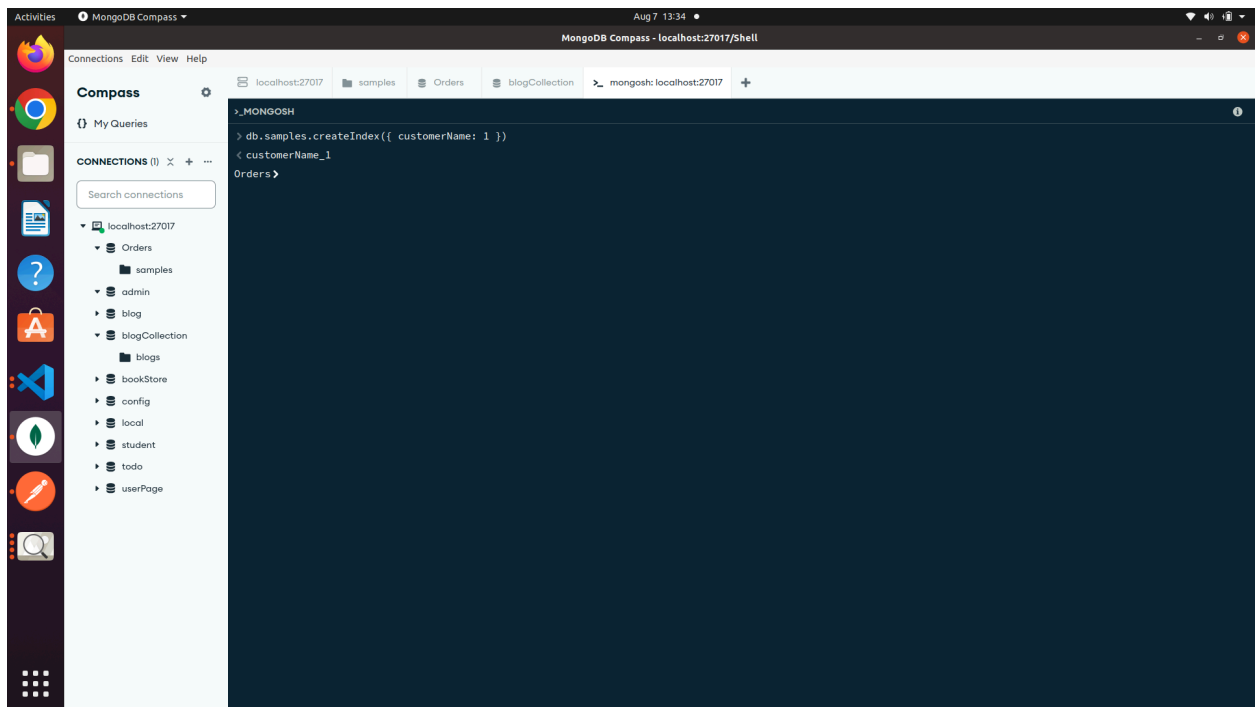


2.Create an index on customerName. Run a query filtering by customerName and check performance using explain("executionStats").



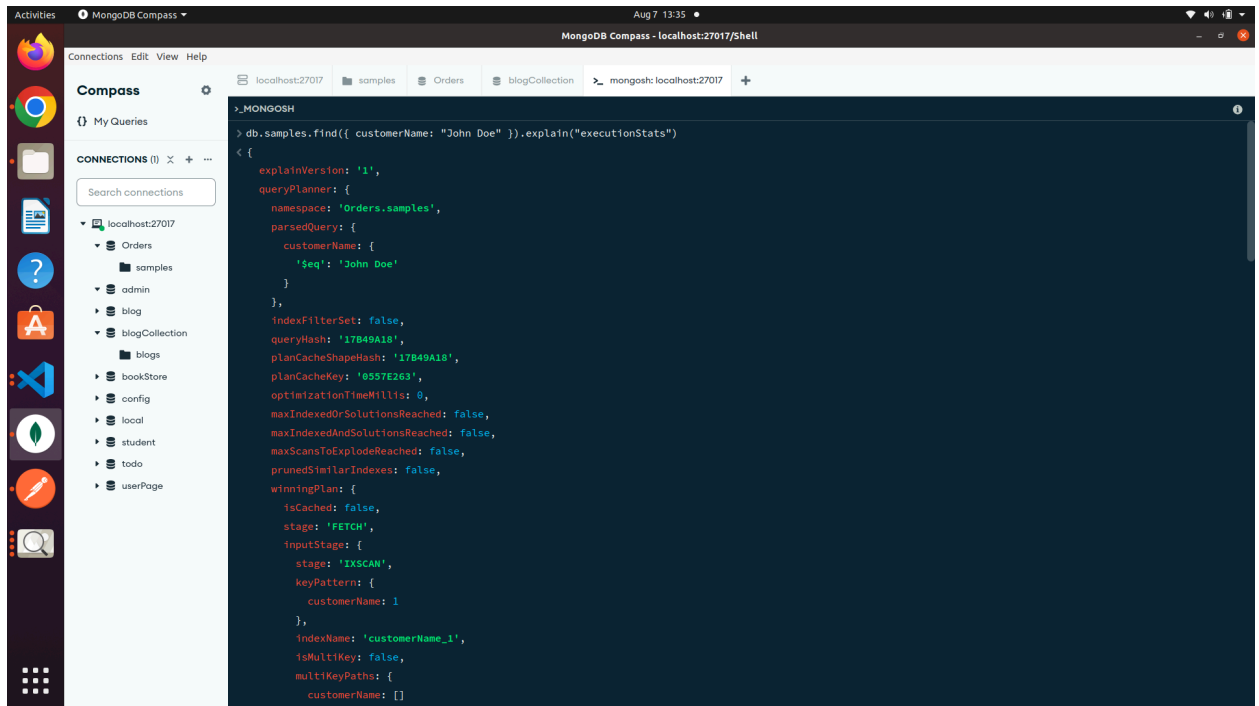
The screenshot shows the MongoDB Compass interface. The left sidebar displays the database structure with collections: Orders, samples, admin, blog, and blogCollection. The main panel shows a query in the MONGODB shell: `db.samples.find({ customerName: "John Doe" }).explain("executionStats")`. The output displays the execution plan, indicating a COLLSCAN stage with a filter on customerName.

```
> db.samples.find({ customerName: "John Doe" }).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Orders.samples',
    parsedQuery: {
      customerName: {
        '$eq': 'John Doe'
      }
    },
    indexFilterSet: false,
    queryHash: '17849A18',
    planCacheShapeHash: '17849A18',
    planCacheKey: '56634888',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExploreReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        customerName: {
          '$eq': 'John Doe'
        }
      },
      direction: 'forward'
    },
    rejectedPlans: []
  },
```

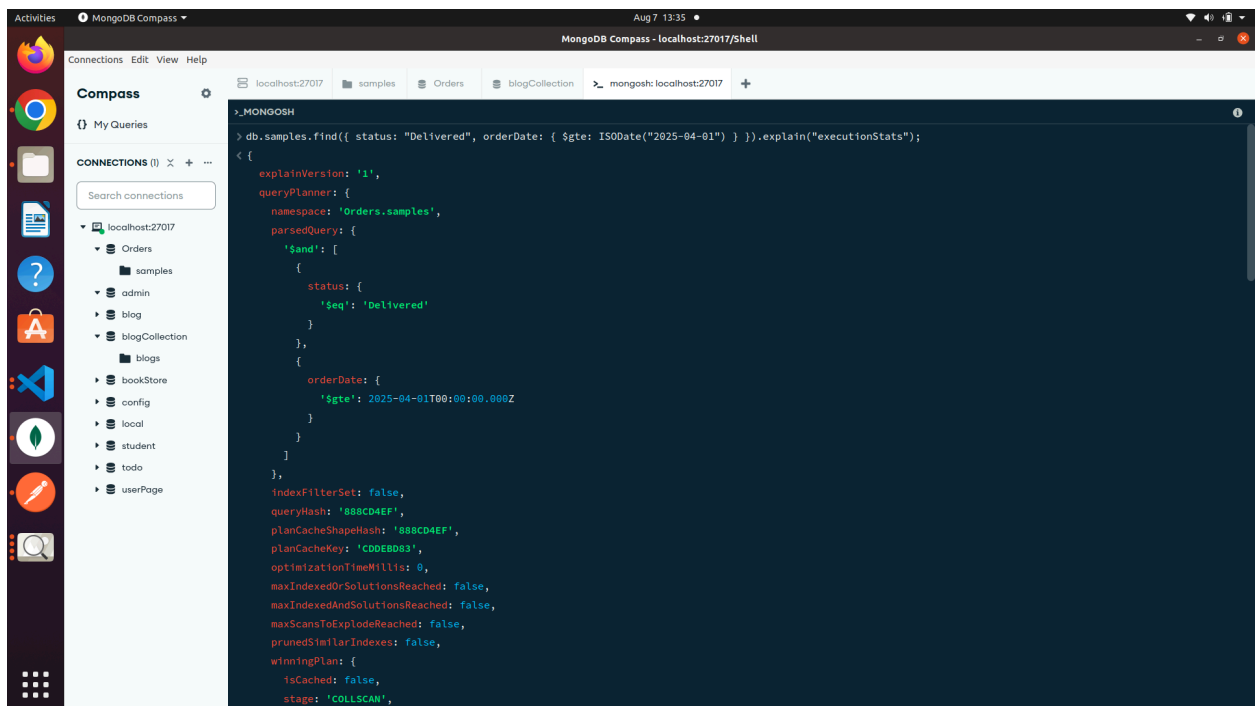


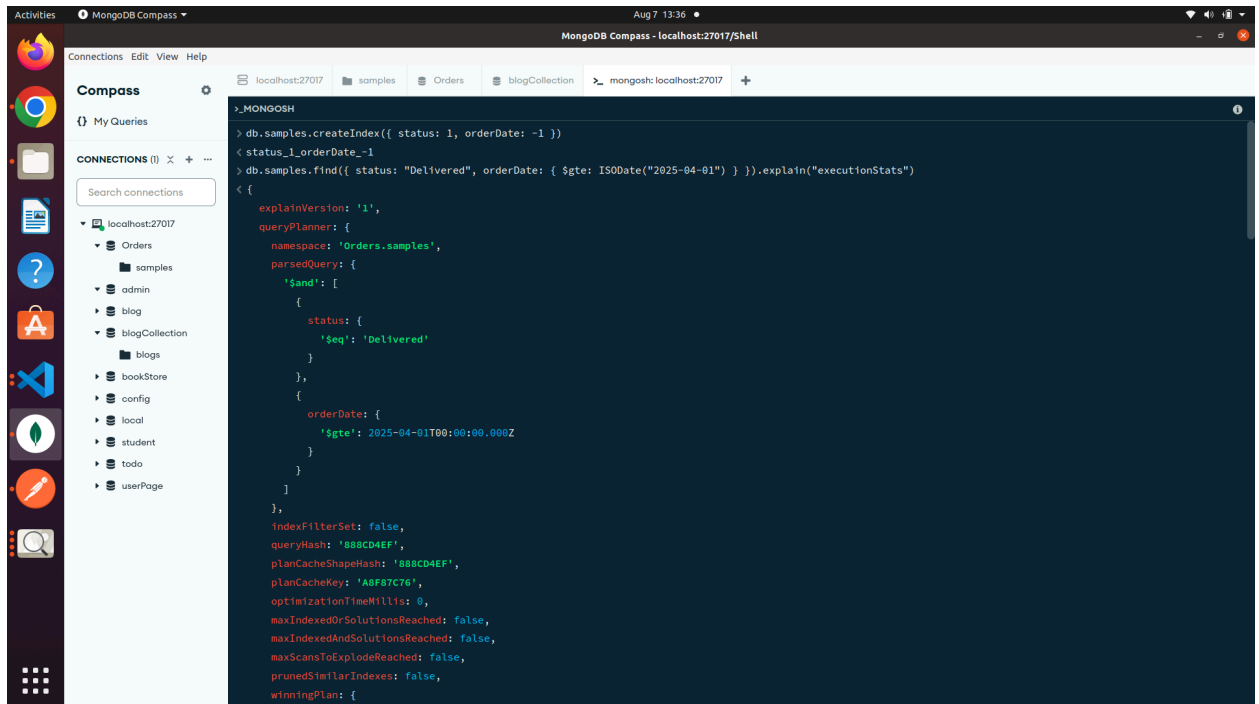
The screenshot shows the MongoDB Compass interface after creating an index. The main panel shows the command: `db.samples.createIndex({ customerName: 1 })`. The output confirms the index creation: `< customerName_1` and `Orders`.

```
> db.samples.createIndex({ customerName: 1 })
< customerName_1
Orders
```

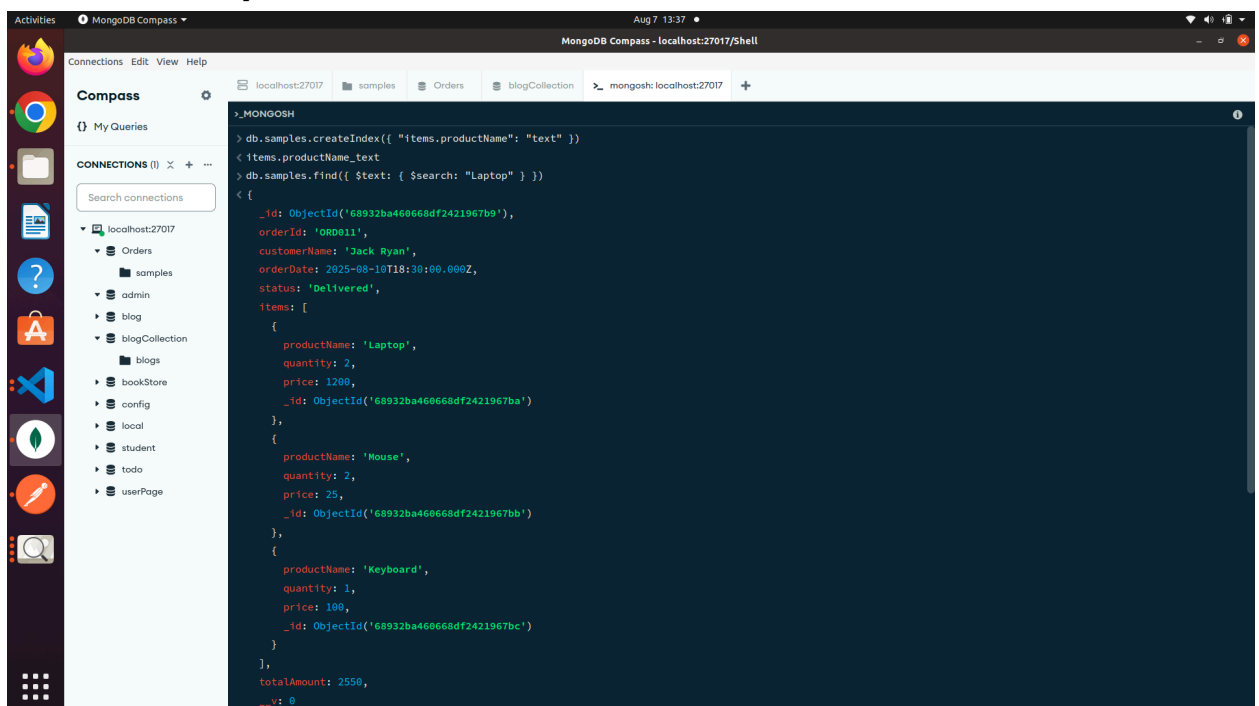


3. Create a compound index on status and orderDate. Run a query filtering by both and compare performance (before vs after).

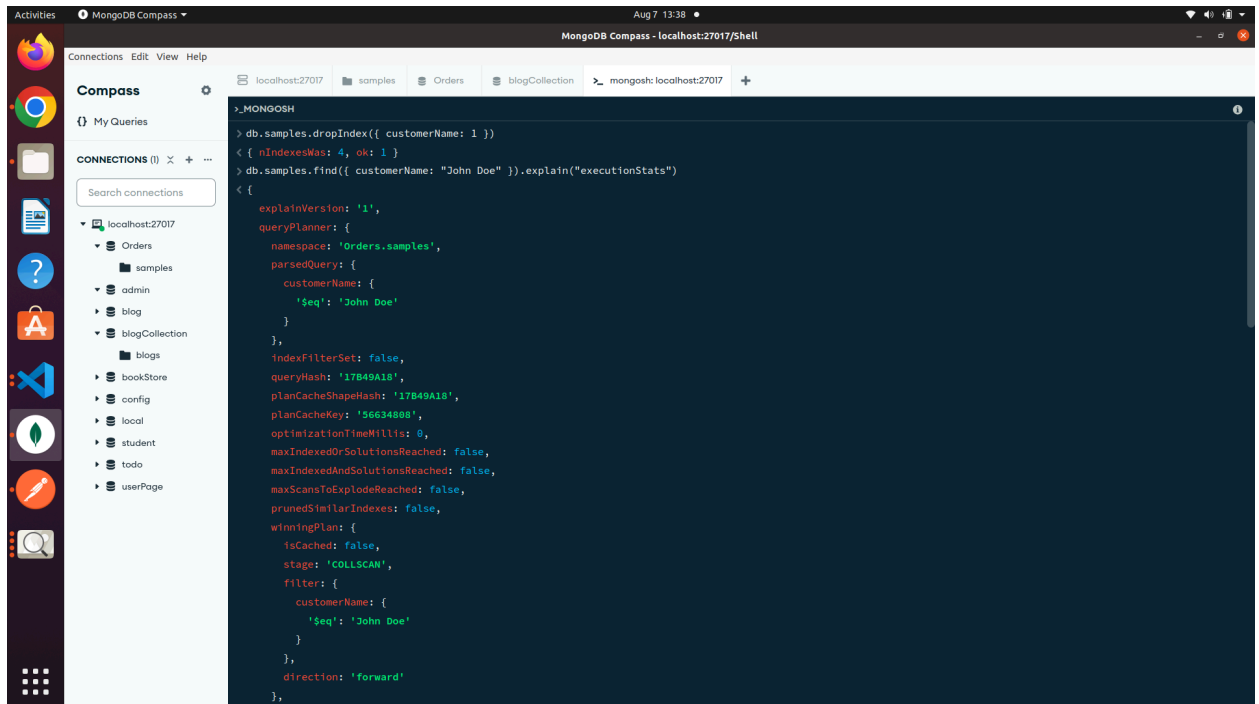




4. Create a text index on items.productName. Perform a text search for a product.



5.Drop an index and observe performance difference.



The screenshot shows the MongoDB Compass application interface. On the left, the 'Connections' panel lists the local connection 'localhost:27017' with a tree view of databases including 'Orders', 'samples', 'admin', 'blog', 'blogCollection', 'blogs', 'bookStore', 'config', 'local', 'student', 'todo', and 'userPage'. The main panel is titled 'MongoDB Compass - localhost:27017/Shell' and contains a MongoDB shell session. The session shows the execution of the command `db.samples.dropIndex({ customerName: 1 })`, which returns `< { nIndexesWas: 4, ok: 1 }`. This is followed by `db.samples.find({ customerName: "John Doe" }).explain("executionStats")`, which returns a detailed execution plan. The plan indicates a 'COLLSCAN' stage, meaning the database is scanning all documents in the collection because the index has been dropped.

```
>_MONGOSH
> db.samples.dropIndex({ customerName: 1 })
< { nIndexesWas: 4, ok: 1 }
> db.samples.find({ customerName: "John Doe" }).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Orders.samples',
    parsedQuery: {
      customerName: {
        '$eq': 'John Doe'
      }
    },
    indexFilterSet: false,
    queryHash: '17B49A18',
    planCacheShapeHash: '17B49A18',
    planCacheKey: '56634888',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        customerName: {
          '$eq': 'John Doe'
        }
      },
      direction: 'forward'
    },
  },
}
```